# Tugas Besar 1 : Searching

## IF3170 Inteligensi Artifisial 2026/2027

## FrontierFolly




Anggota
Mohammad Najmutstsaqib 13223040
Ega Luthfi Rais 13524115
Ahmad Fauzan Putra 13524141

# I. Deskripsi Persoalan

Tugas ini bertujuan untuk membuat konfigurasi pengisian paket paling optimal untuk sebuah truk peti kemas. Optimal disini berarti **memaksimalkan** fungsi objektif.

### Daftar Variabel Permasalahan

| Entitas | Atribut | Contoh | Keterangan |
|---|---|---|---|
| Package | id | "RusdiPackage" | *Identifier* |
| | Dimensions | {w:3, l:4, h:1} | Ukuran sebuah paket, yaitu *width*, *length*, dan *height* |
| | Orientation | {w:x, l:y, h:z} | Sumbu yang sejajar dengan setiap sisi paket. Terdapat enam kemungkinan orientasi, misalnya {w:y, l:x, h:z} setelah rotasi 90° terhadap sumbu *z*. |
| | Value | 60 | Nilai harga sebuah paket (satuan unit) |
| | Weight | 5 | Berat paket (satuan unit berat) |
| | IsFragile | False | Benda pecah-belah |
| | ETA | 3 | Estimated Time of Arrival, perkiraan waktu kedatangan paket (dalam sisa hari) |
| Truck | Dimensions | {w:10, l:20, h:5} | Ukuran sebuah peti kemas pada truk, yaitu *width*, *length*, dan *height* |
| | MaxCapacity | 500 | Kapasitas angkut maksimum pada suatu peti kemas (satuan unit berat) |

### Ketentuan Representasi

Berikut prosedur bagaimana dalam merepresentasikan permasalahan ini:

- Paket hanya berbentuk kubus / balok. Paket dapat berbeda ukuran satu sama lain dan tidak semua harus dimasukkan ke dalam peti kemas pada truk sehingga suatu paket dapat berada di luar **(seluruh bagian paket di luar)**
- Koordinat berupa bilangan bulat. Cara representasi koordinat dibebaskan.
- Struktur data yang digunakan untuk representasi dibebaskan. Misalnya menggunakan *key-value* seperti JSON.
- Inisialisasi (***initial state***) random.
- Langkah neighbor (***move***) yang diperbolehkan tiap iterasi:

1. Menukar dua paket, baik di dalam maupun di luar truk
2. Memindahkan sebuah paket ke suatu koordinat yang berbeda
3. Merotasi paket 90° terhadap sumbu *x*, *y*, atau *z*

- Representasi solusi (***final state***) adalah pemetaan semua paket yang ada dan juga pemetaan semua koordinat dalam peti kemas pada truk. Secara detail yang harus dapat ditunjukkan oleh state dan visualisasi adalah:
    1. Bagaimana keadaan semua paket yang di dalam maupun yang di luar
    2. Bagaimana keadaan peti kemas pada truk
    3. *Value* pada *state* tersebut

## Objective Function & Constraints

Agar permasalahan ini tidak menjadi kompleks, berikut arahan yang diberikan

- **Objective function**

  Permasalahan ini bertujuan untuk memaksimalkan *value* peti kemas pada truk.

- **Kardus tidak bisa melayang dan menembus dinding peti kemas**

  Melayang disini didefinisikan dengan misalkan paket $i$ setelah orientasi tertentu memiliki posisi sudut kiri-bawah-belakang

  $$(x_i\ , y_i\ , z_i)$$

  dan dimensi

  $$(w_i\ , d_i\ , h_i)$$

  sehingga alas paket $i$ adalah

  $$B_i = \{(x, y, z_i) \mid x_i \leq x < x_i{+}w_i,\ y_i \leq y < y_i{+}d_i\ \})$$

  Sebuah titik di alas memiliki *support* apabila:

  1. $Z_{i\ =\ 0}$, artinya di alas truk, atau
  2. Terdapat paket lain $j$ yang mempunyai bagian atas tepat di bawah titik tersebut

  Maka melayang adalah kondisi tidak ada satupun titik alas yang memiliki support.

- **Barang pecah-belah tidak boleh di bawah paket lain**

  Barang pecah-belah tidak boleh menjadi *support*

- **Truk tidak boleh mengangkut melebihi kapasitasnya**

# II. Pembahasan

Bagian ini menjelaskan implementasi algoritma local search beserta hasil eksperimennya. Seluruh program ditulis dalam bahasa Python dan tersimpan pada folder src repositori. Data uji menggunakan src/data/packages.json yang berisi 26 paket (total value 3420, total berat 558, sembilan di antaranya fragile) dan peti kemas berukuran 10 × 20 × 5 dengan kapasitas angkut 500. Struktur umum program adalah:

representasi state, pembangkitan initial state acak yang selalu valid, pencarian solusi dengan algoritma local search, serta visualisasi state dan nilai objective.

Empat varian Hill Climbing diimplementasikan pada src/hill_climbing/hill_climbing.py (steepest ascent, stochastic, sideways move, dan random restart), sedangkan Simulated Annealing diimplementasikan pada src/simulated_annealing/sa.py. Setiap varian dijalankan tiga kali dengan seed 1–3 pada data yang sama dan seluruh hasilnya disimpan pada folder output repositori.

### Representasi State dan Fungsi Objektif

State merepresentasikan posisi seluruh paket dan disimpan sebagai dictionary JSON dengan kunci id paket. Setiap paket dipetakan ke posisi sudut kiri-bawah-belakang (x, y, z) berupa bilangan bulat beserta orientasinya; paket yang tidak dimuat ke peti kemas diberi posisi null sehingga seluruh paket (di dalam maupun di luar truk) tetap terdefinisi pada state. Orientasi direpresentasikan sebagai permutasi (0, 1, 2) yang memetakan sumbu dasar paket (width, length, height) ke sumbu truk (x, y, z); keenam permutasi merepresentasikan enam orientasi yang diperbolehkan. Contoh potongan state:

```
{
  "Z": {"position": [3, 14, 0], "orientation": [0, 1, 2]},
  "K": {"position": null,       "orientation": [2, 1, 0]}
}
```

Fungsi objektif yang dimaksimalkan adalah total value paket yang berada di dalam truk. packageScore memetakan sebuah paket ke skornya sesuai objective yang dipilih (value pada eksperimen utama, urgency pada bonus), sedangkan stateScore menjumlahkan skor seluruh paket yang berposisi tidak null:

*src/hill_climbing/hill_climbing.py — packageScore dan stateScore*

```
def packageScore(package, objective="value"):
    if objective == "value":
        return package["value"]
    if objective == "urgency":
        # Bonus
        return package["value"] / package["eta"]
    raise ValueError(f"Objective '{objective}' tidak dikenali")

def stateScore(state, packages, objective="value"):
    return sum(
        packageScore(packages[k], objective)
        for k, placement in state.items()
        if placement["position"] is not None
    )
```

### Constraint State (state_valid)

Seluruh kandidat solusi divalidasi oleh fungsi state_valid yang menjalankan enam pengecekan: (1) check_orientation memastikan orientasi berupa permutasi yang sah; (2) check_bounds memastikan tidak ada bagian paket yang menembus dinding peti kemas; (3) check_collision memastikan tidak ada dua paket yang saling menembus, diperiksa dengan uji tumpang-tindih pada ketiga sumbu; (4) check_weight membatasi total berat muatan agar tidak melebihi max_capacity; (5) check_support melarang

paket melayang, yaitu paket dengan z > 0 wajib memiliki paket lain yang permukaan atasnya tepat berada di bawah alas paket tersebut; dan (6) check_fragile melarang barang pecah-belah menjadi penopang maupun berada di bawah paket lain.

Berikut implementasi pengecekan support dan fragile beserta fungsi pembungkusnya. check_bounds, check_collision, dan check_weight merupakan pengecekan geometris sederhana yang tersedia lengkap pada src/constraint.py.

*src/constraint.py — contact, check_support, check_fragile, state_valid*

```
def contact(pos_a, pos_b, rotate_dims_a, rotate_dims_b):
    # Apakah permukaan atas package a menyentuh permukaan bawah package b
    z_contact = (pos_a[2] + rotate_dims_a[2] == pos_b[2])

    # Apakah kedua paket beririsan di sumbu x
    x_contact = (
        (pos_a[0] < (pos_b[0] + rotate_dims_b[0])) and
        (pos_b[0] < (pos_a[0] + rotate_dims_a[0]))
    )

    # Apakah kedua paket beririsan di sumbu x
    y_contact = (
        (pos_a[1] < (pos_b[1] + rotate_dims_b[1])) and
        (pos_b[1] < (pos_a[1] + rotate_dims_a[1]))
    )

    # Apakah ada contact
    return z_contact and x_contact and y_contact

def check_support(state, packages):
    list_state_id = list(state.keys()) # Semua list key dari state
    state_len = len(state)
    for i in range(state_len): # https://www.geeksforgeeks.org/python/get-length-of-dictionary-
in-python/
        package_1 = list_state_id[i]
        pos_1 = state[package_1]["position"]
        if (pos_1 is None) or (pos_1[2] == 0): # check apakah package 1 tidak di dalam truk atau
berada di lantai truk
            continue

        # Dimensi paket
        dims_1 = packages[package_1]["dimensions"]

        # Orientasi paket
        ori_1 = state[package_1]["orientation"]

        # Dimensi setelah rotasi
        rotate_dims_1 = [dims_1[k] for k in ori_1]

        supported = False
        for j in range(state_len):
            package_2 = list_state_id[j]
            pos_2 = state[package_2]["position"]
            if (i == j) or (pos_2 is None): # Jika package 1 dan 2 adalah sama atau package 2
tidak ada di dalam truk
                continue

            # Dimensi paket
            dims_2 = packages[package_2]["dimensions"]

            # Orientasi paket
            ori_2 = state[package_2]["orientation"]
```

```
            # Dimensi setelah rotasi
            rotate_dims_2 = [dims_2[k] for k in ori_2]

            if contact(pos_2, pos_1, rotate_dims_2, rotate_dims_1):
                supported = True
                break

        if not supported:
            return False

    return True

def check_fragile(state, packages):
    list_state_id = list(state.keys()) # Semua list key dari state
    state_len = len(state)
    for i in range(state_len): # https://www.geeksforgeeks.org/python/get-length-of-dictionary-
in-python/
        package_1 = list_state_id[i]
        pos_1 = state[package_1]["position"]
        if (pos_1 is None) or (not packages[package_1]["is_fragile"]): # check apakah package 1
tidak di dalam truk atau bukan fragile (karena sebagai penopang)
            continue

        # Dimensi paket
        dims_1 = packages[package_1]["dimensions"]

        # Orientasi paket
        ori_1 = state[package_1]["orientation"]

        # Dimensi setelah rotasi
        rotate_dims_1 = [dims_1[k] for k in ori_1]

        for j in range(state_len):
            package_2 = list_state_id[j]
            pos_2 = state[package_2]["position"]
            if (i == j) or (pos_2 is None): # Jika package 1 dan 2 adalah sama atau package 2
tidak ada di dalam truk
                continue

            # Dimensi paket
            dims_2 = packages[package_2]["dimensions"]

            # Orientasi paket
            ori_2 = state[package_2]["orientation"]

            # Dimensi setelah rotasi
            rotate_dims_2 = [dims_2[k] for k in ori_2]

            # Jika ada kontak antara fragile (yang dibawah) dengan package lain (yang diatas)
maka False
            if contact(pos_1, pos_2, rotate_dims_1, rotate_dims_2):
                return False

    return True

def state_valid(state, packages, truck):
    return (check_orientation(state) # Orientasi paket valid
            and check_bounds(state, packages, truck) # Aman dari out of bounds
            and check_collision(state, packages) # Aman dari collision
            and check_weight(state, packages, truck) # Berat tidak melebihi maksimum
            and check_support(state, packages) # Aman dari package melayang
            and check_fragile(state, packages))
```

### Inisialisasi State dan Visualisasi

Initial state dibangkitkan acak oleh gen_initial_state: urutan paket dan jumlah paket yang dicoba dimasukkan diacak, demikian pula orientasi dan koordinatnya. Penempatan hanya diterima bila state valid; setiap paket diberi maksimum 50 percobaan dan dibiarkan di luar truk bila gagal. Fungsi possible_height mengumpulkan ketinggian z yang layak untuk penempatan (lantai truk dan permukaan atas paket non-fragile) sehingga paket baru selalu memiliki penopang.

*src/initial_state.py — possible_height dan gen_initial_state*

```
def possible_height(state, packages):
    height = [0]

    for package_id, placement in state.items():
        pos = placement["position"]

        if ((pos is None) or (packages[package_id]["is_fragile"])): # Jika paket berada di luar
atau paket base nya adalah fragile
            continue

        dimensions = packages[package_id]["dimensions"]
        orientation = placement["orientation"]
        rotate_dims = [dimensions[k] for k in orientation]

        top_z = pos[2] + rotate_dims[2] # Ketinggian paket
        height.append(top_z)

    return list(set(height))

def gen_initial_state(packages, truck):
    # Inisialisasi state
    state = {}

    # State di inisialisasi di luar truck
    for package_id in packages:
        state[package_id] = {
            "position" : None,
            "orientation" : (0, 1, 2)
        }

    # Mengacak package_id
    list_package_id = list(packages.keys())
    random.shuffle(list_package_id) # https://www.w3schools.com/python/ref_random_shuffle.asp

    # Randomize jumlah paket yang masuk ke truk
    num_packages = random.randint(1, len(list_package_id))

    truck_dims = truck["dimensions"]

    for package_id in list_package_id[:num_packages]:
        dimensions = packages[package_id]["dimensions"]

        for _ in range(50): # 50 kali pengulangan jika gagal maka paket tidak dimasukkan
            # https://www.w3schools.com/python/ref_random_sample.asp
            orientation = tuple(random.sample([0, 1, 2], k=3)) # Mengacak orientasi
            rotate_dims = [dimensions[i] for i in orientation]

            # https://www.w3schools.com/python/ref_func_any.asp
            if any(rotate_dims[k] > truck_dims[k] for k in range(3)): # Jika ukuran paket lebih
besar dari dimensi truck nya
```

```
                continue

            # Ambil ketinggian yang tidak melebihi batas dimensi truck
            heights = [
                z for z in possible_height(state, packages)
                if z + rotate_dims[2] <= truck_dims[2]
            ]

            # Jika possible ketinggian tidak ditemukan
            if not heights:
                continue

            # Mengacak koordinat x, y, dan z
            position = (
                random.randint(0, truck_dims[0] - rotate_dims[0]),  # X
                random.randint(0, truck_dims[1] - rotate_dims[1]),  # Y
                random.choice(heights)                              # Z
            )

            state[package_id] = {
                "position": position,
                "orientation": orientation
            }

            # Jika seluruh constraint valid
            if state_valid(state, packages, truck):
                break

            # Jika tidak valid, paket dikembalikan ke luar
            state[package_id]["position"] = None

    return state
```

Visualisasi dibuat dengan matplotlib: plotState menggambar setiap paket sebagai balok tiga dimensi di dalam kerangka peti kemas, sedangkan plotValues menggambar kurva nilai objective terhadap iterasi. Contoh keluaran kedua fungsi ditunjukkan pada gambar-gambar di subbab berikut.

*src/visual.py — plotState*

```
def plotState(state, packages, truck, path):
    W, L, H = truck["dimensions"]
    a = plt.figure(figsize=(4,4)).add_subplot(projection="3d")
    faces, colors = [], []
    palette = plt.cm.tab20.colors + plt.cm.tab20b.colors
    for id, x in state.items():
        if x["position"] is None:
            continue
        (px, py, pz), (w, l, h) = x["position"], [packages[id]['dimensions'][i] for i in
x["orientation"]]
        faces += box(px, py, pz, w, l, h)
        colors += [palette[list(packages).index(id)]] * 6
    a.add_collection3d(Poly3DCollection(faces, facecolor=colors, edgecolor="blue"))
    a.add_collection3d(Poly3DCollection(box(0, 0, 0, W, L, H), facecolor=(0, 0, 0, 0),
edgecolor="black"))
    a.set(xlim=(0, W), ylim=(0, L), zlim=(0, H), xlabel="x", ylabel="y", zlabel="z")
    a.set_box_aspect((W, L, H), zoom=1)
    a.grid(False)
    for x in (a.xaxis, a.yaxis, a.zaxis):
        x.set_pane_color("white")
    plt.savefig(path, dpi=100)
    plt.close()
```

### Hill Climbing — Steepest Ascent

Steepest ascent memilih langkah menaik terbaik pada setiap iterasi dan berhenti ketika tidak ada lagi langkah yang meningkatkan nilai. Dua jenis langkah dipertimbangkan: menambahkan paket dari luar ke posisi yang layak, dan menukar paket di dalam truk dengan paket luar yang lebih bernilai pada slot yang sama. Penempatan paket menggunakan heuristik deterministik: kandidat koordinat x dan y diambil dari sisi paket-paket lain dan dinding peti kemas (axisCandidates), kandidat z diambil dari possible_height, lalu seluruh kombinasi diuji dengan orientasi yang muat. Heuristik ini mereduksi ruang penempatan kontinu menjadi himpunan titik kandidat terbatas sehingga evaluasi setiap langkah menjadi cepat dan konsisten.

*src/hill_climbing/hill_climbing.py — axisCandidates, findFeasiblePlacement, bestImprovingMove, steepestAscent*

```
def axisCandidates(boxes, axis, size, limit):
    values = {0, limit - size}
    for position, dims, _ in boxes:
        near = position[axis]
        far = near + dims[axis]
        values.update((near, far, near - size, far - size))
    return sorted(v for v in values if 0 <= v <= limit - size)

def findFeasiblePlacement(state, package_id, packages, truck, exclude=None):
    W, L, H = truck["dimensions"]
    fragile = packages[package_id]["is_fragile"]

    if totalWeight(state, packages) + packages[package_id]["weight"] > truck["max_capacity"]:
        return None

    boxes = insideBoxes(state, packages)
    heights = possible_height(state, packages)

    for orientation in ORIENTATIONS:
        w, l, h = rotatedDims(package_id, orientation, packages)
        if w > W or l > L or h > H:
            continue

        xs = axisCandidates(boxes, 0, w, W)
        ys = axisCandidates(boxes, 1, l, L)
        zs = sorted(z for z in heights if z + h <= H)
        if not xs or not ys or not zs:
            continue

        for z in zs:
            # Overlap z
            rel = [
                (p[0], p[1], p[2], d[0], d[1], d[2], f)
                for p, d, f in boxes
                if (p[2] < z + h and z < p[2] + d[2]) or p[2] + d[2] == z or p[2] == z + h
            ]

            for x in xs:
                for y in ys:
                    if exclude is not None and (x, y, z) == exclude:
                        continue

                    valid = True
                    supported = z == 0
                    for kx, ky, kz, kw, kl, kh, k_fragile in rel:
                        if not (overlap1D(x, w, kx, kw) and overlap1D(y, l, ky, kl)):
                            continue
```

```
                        if kz < z + h and z < kz + kh:
                            valid = False
                            break
                        if kz + kh == z:
                            if k_fragile:  # fragile tidak boleh menjadi penopang
                                valid = False
                                break
                            supported = True
                        elif kz == z + h and fragile:
                            valid = False
                            break

                    if valid and supported:
                        return (x, y, z), orientation

    return None

def bestImprovingMove(state, packages, truck, objective="value"):
    scores = {k: packageScore(packages[k], objective) for k in packages}
    inside = [k for k, placement in state.items() if placement["position"] is not None]
    outside = [k for k, placement in state.items() if placement["position"] is None]

    best = None

    # highest delta
    for j in sorted(outside, key=lambda k: (-scores[k], k)):
        if scores[j] <= 0:
            break
        placement = findFeasiblePlacement(state, j, packages, truck)
        if placement is None:
            continue
        position, orientation = placement
        new = fotokopi(state)
        new[j] = {"position": position, "orientation": orientation}
        if state_valid(new, packages, truck):
            best = (scores[j], new, f"tambah paket {j} ke {position} orientasi {orientation}")
            break

    pairs = [
        (scores[j] - scores[i], i, j)
        for i in inside
        for j in outside
        if scores[j] > scores[i]
    ]
    pairs.sort(key=lambda pair: (-pair[0], pair[1], pair[2]))

    for delta, i, j in pairs:
        if best is not None and delta <= best[0]:
            break
        slot = state[i]["position"]
        reduced = fotokopi(state)
        reduced[i] = {"position": None, "orientation": state[i]["orientation"]}
        for orientation in ORIENTATIONS:
            new = fotokopi(reduced)
            new[j] = {"position": slot, "orientation": orientation}
            if state_valid(new, packages, truck):
                best = (delta, new, f"swap paket dalam {i} dengan paket luar {j}")
                break

    return best

def steepestAscent(packages, truck, objective="value", maxIter=2000, seed=None):
    if seed is not None:
        random.seed(seed)

    start = time.perf_counter()
    initial = gen_initial_state(packages, truck)
```

```
    current = fotokopi(initial)
    current_value = stateScore(current, packages, objective)

    values = [current_value]
    iterations = 0

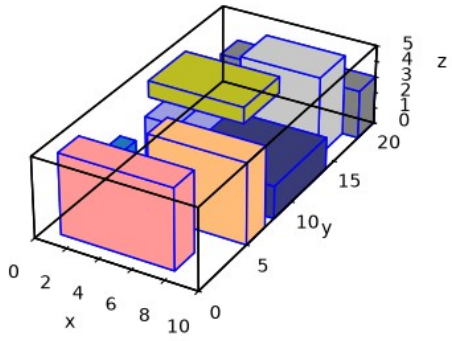
    while iterations < maxIter:
        best = bestImprovingMove(current, packages, truck, objective)
        if best is None:
            break
        _, new_state, _ = best
        current = new_state
        current_value = stateScore(current, packages, objective)
        values.append(current_value)
        iterations += 1

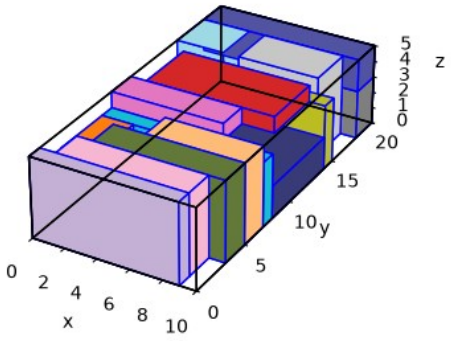
    return {
        "variant": "steepest_ascent",
        "objective": objective,
        "seed": seed,
        "initial": initial,
        "final": current,
        "finalVal": current_value,
        "realValue": rawValue(current, packages),
        "duration": time.perf_counter() - start,
        "valuesOT": values,
        "iterations": iterations,
        "max_iter": maxIter
    }
```

Hasil tiga run (seed 1–3) beserta durasi dan banyak iterasi hingga pencarian berhenti:

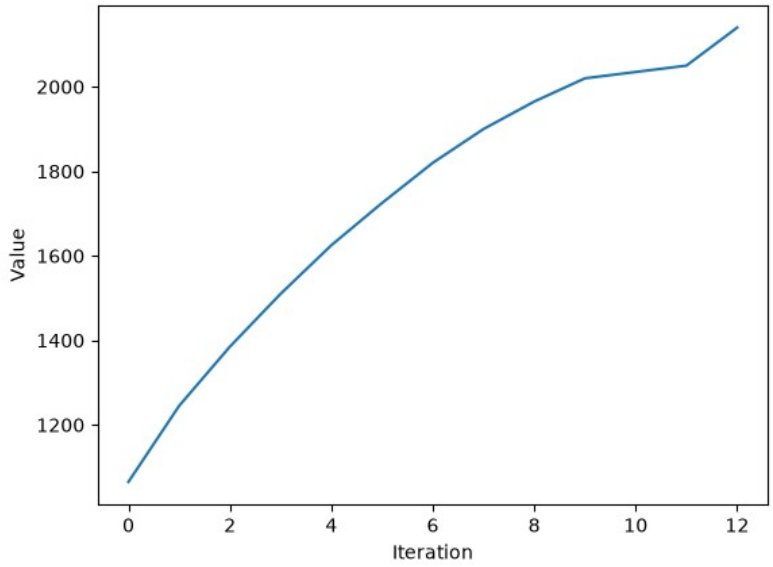
*Tabel 1. Hasil eksperimen steepest ascent hill climbing.*

| Run (seed) | Nilai akhir | Durasi (detik) | Banyak iterasi |
|---|---|---|---|
| 1 | 2035 | 0,039 | 13 |
| 2 | 2140 | 0,122 | 12 |
| 3 | 1815 | 0,136 | 6 |



*Gambar 1. State awal (kiri) dan state akhir (kanan) steepest ascent run 2 (nilai 2140).*




*Gambar 2. Nilai objective terhadap iterasi steepest ascent run 2.*

Run 3 berhenti hanya dalam 6 iterasi dan seluruh run selesai di bawah 0,15 detik karena setiap iterasi hanya mengevaluasi kandidat deterministik. Nilai akhir bervariasi cukup besar (1815–2140) — karakteristik khas steepest ascent yang tidak dapat keluar dari local optimum, sehingga kualitas solusinya sangat bergantung pada initial state acak.

### Hill Climbing — Stochastic

Stochastic hill climbing memilih neighbor secara acak melalui randomNeighbor yang mendukung lima jenis langkah sesuai spesifikasi: swap dua paket (di dalam maupun di luar truk), add, remove, move, dan rotate. Langkah pertama yang menaikkan nilai langsung diterima; setiap iterasi memberi kesempatan sampai 100 percobaan, dan pencarian berhenti ketika tidak ada perbaikan dalam satu putaran penuh. Pendekatan ini jauh lebih murah per iterasi dibanding steepest ascent, dengan konsekuensi kualitas solusi biasanya lebih rendah.

*src/hill_climbing/hill_climbing.py — randomNeighbor dan stochastic*

```
def randomNeighbor(state, packages, truck, t=50):
    W, L, H = truck["dimensions"]
    inside = [k for k, placement in state.items() if placement["position"] is not None]
    outside = [k for k, placement in state.items() if placement["position"] is None]
    if not inside and not outside:
        return None

    move = random.choice(["swap", "add", "remove", "move", "rotate"])

    if move == "swap":
        if len(packages) < 2:
            return None
        a, b = random.sample(list(packages), 2)
```

```
        if state[a]["position"] is None and state[b]["position"] is None:
            return None
        new = fotokopi(state)
        new[a]["position"], new[b]["position"] = state[b]["position"], state[a]["position"]
        if state_valid(new, packages, truck):
            return new
        return None

    if move == "add" and outside:
        j = random.choice(outside)
        for _ in range(t):
            orientation = random.choice(ORIENTATIONS)
            w, l, h = rotatedDims(j, orientation, packages)
            if w > W or l > L or h > H:
                continue
            heights = [z for z in possible_height(state, packages) if z + h <= H]
            if not heights:
                continue
            position = (random.randint(0, W - w), random.randint(0, L - l),
random.choice(heights))
            new = fotokopi(state)
            new[j] = {"position": position, "orientation": orientation}
            if state_valid(new, packages, truck):
                return new
        return None

    if move == "remove" and inside:
        i = random.choice(inside)
        new = fotokopi(state)
        new[i]["position"] = None
        if state_valid(new, packages, truck):
            return new
        return None

    if move == "move" and inside:
        j = random.choice(inside)
        orientation = state[j]["orientation"]
        w, l, h = rotatedDims(j, orientation, packages)
        reduced = fotokopi(state)
        reduced[j]["position"] = None
        for _ in range(t):
            heights = [z for z in possible_height(reduced, packages) if z + h <= H]
            if not heights:
                continue
            position = (random.randint(0, W - w), random.randint(0, L - l),
random.choice(heights))
            if position == state[j]["position"]:
                continue
            new = fotokopi(reduced)
            new[j] = {"position": position, "orientation": orientation}
            if state_valid(new, packages, truck):
                return new
        return None

    if move == "rotate" and inside:
        j = random.choice(inside)
        current_ori = state[j]["orientation"]
        others = [o for o in ORIENTATIONS if o != current_ori]
        random.shuffle(others)
        for orientation in others:
            new = fotokopi(state)
            new[j]["orientation"] = orientation
            if state_valid(new, packages, truck):
                return new
        return None

    return None

def stochastic(packages, truck, objective="value", trials=100, maxIter=2000, seed=None):
```

```
    if seed is not None:
        random.seed(seed)

    start = time.perf_counter()
    initial = gen_initial_state(packages, truck)
    current = fotokopi(initial)
    current_value = stateScore(current, packages, objective)

    values = [current_value]
    iterations = 0
    total_attempts = 0

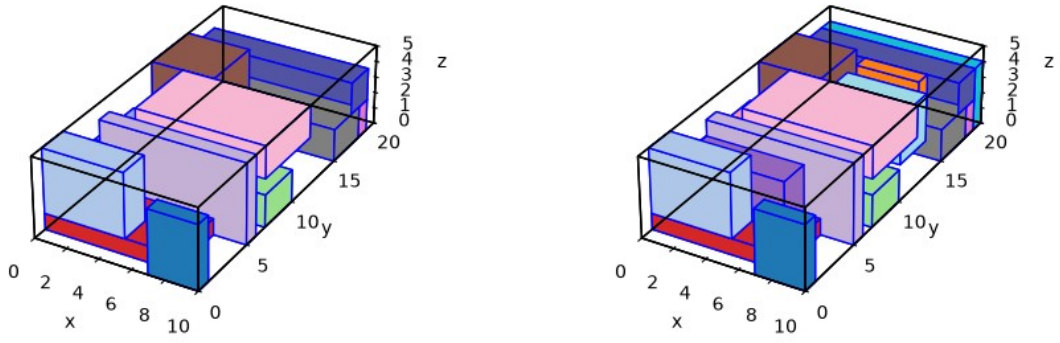
    while iterations < maxIter:
        improved = False
        for _ in range(trials):
            total_attempts += 1
            new_state = randomNeighbor(current, packages, truck)
            if new_state is None:
                continue
            new_value = stateScore(new_state, packages, objective)
            if new_value > current_value:
                current, current_value = new_state, new_value
                values.append(current_value)
                iterations += 1
                improved = True
                break
        if not improved:
            break

    return {
        "variant": "stochastic",
        "objective": objective,
        "seed": seed,
        "initial": initial,
        "final": current,
        "finalVal": current_value,
        "realValue": rawValue(current, packages),
        "duration": time.perf_counter() - start,
        "valuesOT": values,
        "iterations": iterations,
        "max_iter": maxIter,
        "trials_per_iteration": trials,
        "total_attempts": total_attempts
    }
```

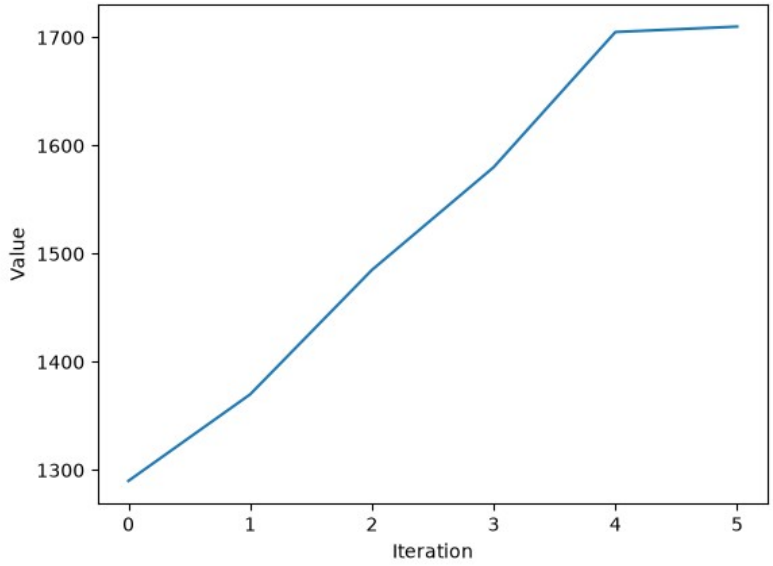
Hasil tiga run:

*Tabel 2. Hasil eksperimen stochastic hill climbing (100 percobaan per iterasi).*

| **Run (seed)** | **Nilai akhir** | **Durasi (detik)** | **Banyak iterasi** |
|---|---|---|---|
| 1 | 1550 | 0,296 | 15 |
| 2 | 1610 | 0,196 | 6 |
| 3 | 1710 | 0,276 | 5 |




*Gambar 3. State awal (kiri) dan state akhir (kanan) stochastic hill climbing run 3 (nilai 1710).*




*Gambar 4. Nilai objective terhadap iterasi stochastic hill climbing run 3.*

Ketiga run berhenti sangat cepat (hanya 5–15 perbaikan yang diterima) karena begitu 100 percobaan berturut-turut gagal, pencarian berakhir. Nilai akhir 1550–1710 lebih rendah daripada steepest ascent dan random restart: meskipun murah, anggaran eksplorasi yang kecil membuat varian ini paling rentan terjebak pada local optimum pertama. Memperbesar jumlah percobaan per iterasi dapat meningkatkan hasil dengan konsekuensi durasi bertambah.

### Hill Climbing — Sideways Move

Varian sideways move dirancang untuk keluar dari plateau dengan menerima langkah yang nilainya sama dengan nilai saat ini. findSidewaysMove mencoba empat jenis

langkah: menukar posisi dua paket di dalam truk (beserta kombinasi orientasinya), menukar paket dalam dengan paket luar yang berskor sama, merotasi paket selama dimensi efektifnya berubah, dan memindahkan paket ke posisi layak lain. Agar tidak berputar-putar, kandidat wajib mengubah geometri state (dicek melalui geometryKey) dan berbeda dari state sebelumnya. Parameter maxSideways membatasi banyak langkah sideways; pada eksperimen ini digunakan 200.

*src/hill_climbing/hill_climbing.py — findSidewaysMove dan sidewaysMove*

```
def findSidewaysMove(state, packages, truck, objective="value", prev_key=None):
    scores = {k: packageScore(packages[k], objective) for k in packages}
    inside = sorted(k for k, placement in state.items() if placement["position"] is not None)
    outside = sorted(k for k, placement in state.items() if placement["position"] is None)
    current_key = geometryKey(state, packages)

    def acceptable(candidate):
        if candidate is None:
            return False
        key = geometryKey(candidate, packages)
        return key != current_key and (prev_key is None or key != prev_key)

    for a, b in itertools.combinations(inside, 2):
        candidate = trySwapPositions(state, a, b, packages, truck)
        if acceptable(candidate):
            return candidate, f"swap posisi {a} <-> {b}"

    for i in inside:
        for j in outside:
            if scores[j] != scores[i]:
                continue
            candidate = trySwapPositions(state, i, j, packages, truck)
            if acceptable(candidate):
                return candidate, f"swap posisi {i} <-> {j}"

    for k in inside:
        current_ori = state[k]["orientation"]
        current_dims = tuple(rotatedDims(k, current_ori, packages))
        for orientation in ORIENTATIONS:
            if orientation == current_ori:
                continue
            if tuple(rotatedDims(k, orientation, packages)) == current_dims:
                continue  # rotasi yang tidak mengubah dimensi efektif
            candidate = fotokopi(state)
            candidate[k] = {"position": state[k]["position"], "orientation": orientation}
            if state_valid(candidate, packages, truck) and acceptable(candidate):
                return candidate, f"rotasi paket {k} ke orientasi {orientation}"

    for k in inside:
        current_pos = state[k]["position"]
        reduced = fotokopi(state)
        reduced[k] = {"position": None, "orientation": state[k]["orientation"]}
        placement = findFeasiblePlacement(reduced, k, packages, truck, exclude=current_pos)
        if placement is None:
            continue
        position, orientation = placement
        candidate = fotokopi(reduced)
        candidate[k] = {"position": position, "orientation": orientation}
        if state_valid(candidate, packages, truck) and acceptable(candidate):
            return candidate, f"pindah paket {k} ke {position}"

    return None, None

def sidewaysMove(packages, truck, objective="value", maxSideways=100, maxIter=2000, seed=None):
    if seed is not None:
```

```
        random.seed(seed)

    start = time.perf_counter()
    initial = gen_initial_state(packages, truck)
    current = fotokopi(initial)
    current_value = stateScore(current, packages, objective)

    values = [current_value]
    iterations = 0
    sideways_total = 0
    previous_key = None
    stopped_by = "no_move"

    while iterations < maxIter:
        best = bestImprovingMove(current, packages, truck, objective)
        if best is not None:
            _, new_state, _ = best
            previous_key = geometryKey(current, packages)
            current = new_state
            current_value = stateScore(current, packages, objective)
            values.append(current_value)
            iterations += 1
            continue

        if sideways_total >= maxSideways:
            stopped_by = "max_sideways"
            break

        new_state, _ = findSidewaysMove(current, packages, truck, objective, previous_key)
        if new_state is None:
            stopped_by = "no_move"
            break

        previous_key = geometryKey(current, packages)
        current = new_state
        values.append(current_value)
        iterations += 1
        sideways_total += 1

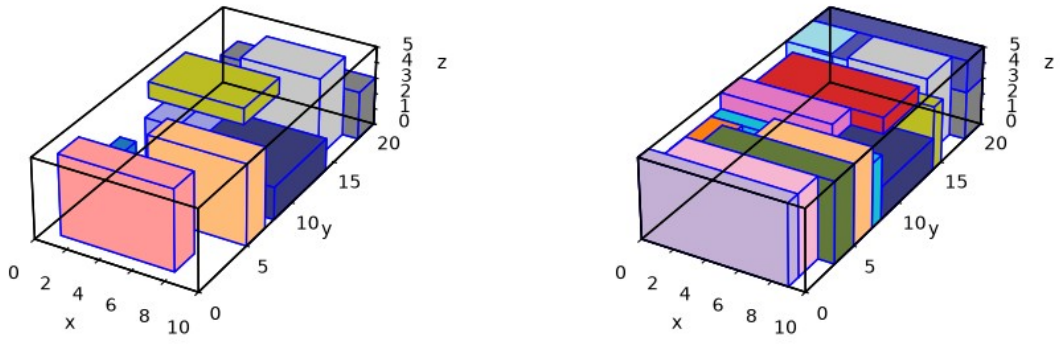
    return {
        "variant": "sideways_move",
        "objective": objective,
        "seed": seed,
        "initial": initial,
        "final": current,
        "finalVal": current_value,
        "realValue": rawValue(current, packages),
        "duration": time.perf_counter() - start,
        "valuesOT": values,
        "iterations": iterations,
        "max_iter": maxIter,
        "max_sideways": maxSideways,
        "sideways_moves": sideways_total,
        "stopped_by": stopped_by
    }
```

Hasil tiga run:

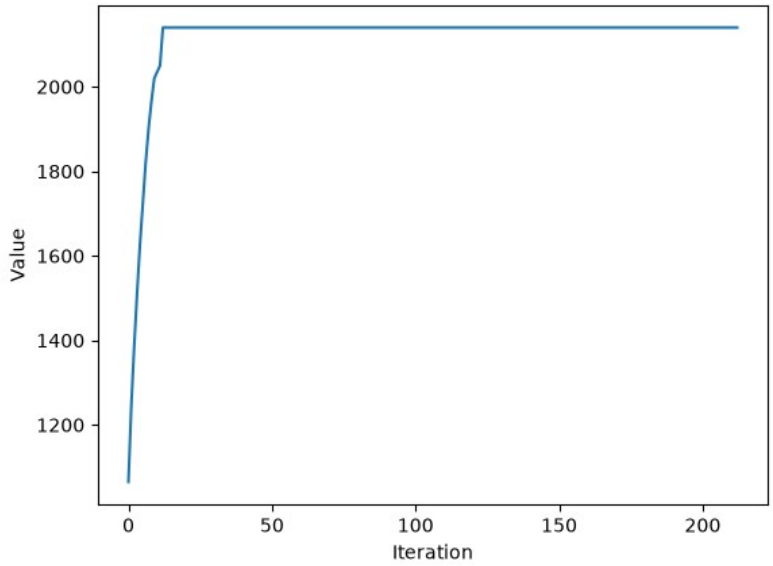
*Tabel 3. Hasil eksperimen hill climbing with sideways move (maxSideways = 200).*

| Run (seed) | Nilai akhir | Durasi (detik) | Banyak iterasi | Sideways move |
|---|---|---|---|---|
| 1 | 2090 | 6,030 | 214 | 200 (batas) |
| 2 | 2140 | 7,677 | 212 | 200 (batas) |

| | | | | |
|---|---|---|---|---|
| 3 | 1815 | 6,108 | 206 | 200 (batas) |




*Gambar 5. State awal (kiri) dan state akhir (kanan) sideways move run 2 (nilai 2140).*




*Gambar 6. Nilai objective terhadap iterasi sideways move run 2.*

Ketiga run berhenti karena batas 200 sideways move tercapai (stopped_by = max_sideways). Hanya run 1 yang berhasil memperbaiki hasil steepest ascent (2035 → 2090); run 2 dan 3 berakhir pada nilai yang sama dengan steepest ascent. Plateau di sekitar local optimum ternyata sempit sehingga langkah sideways sebanyak 200 kali tidak otomatis menemukan lembah yang lebih tinggi. Varian ini juga paling lambat (6–7,7 detik) karena setiap iterasi mengevaluasi banyak kandidat swap-orientasi dan rotasi.

### Hill Climbing — Random Restart

Random restart menjalankan steepest ascent berulang-ulang, masing-masing dari initial state acak baru, lalu menyimpan solusi terbaik dari seluruh restart. Parameter maxRestart (dipakai 20) menghentikan proses ketika jumlah restart maksimum tercapai.

*src/hill_climbing/hill_climbing.py — randomRestart*

```
def randomRestart(packages, truck, objective="value", maxRestart=5, maxIterPerRestart=2000,
seed=None):
    if seed is not None:
        random.seed(seed)

    start = time.perf_counter()
    best_state = None
    best_value = None
    first_initial = None
    iterations_per_restart = []
    restart_values = []
    values = []
    restarts = 0

    while restarts < maxRestart:
        # seed=None: tiap restart mengambil initial state random baru dari RNG global
        result = steepestAscent(packages, truck, objective, maxIter=maxIterPerRestart, seed=None)
        if first_initial is None:
            first_initial = result["initial"]
        restarts += 1
        iterations_per_restart.append(result["iterations"])
        restart_values.append(result["finalVal"])
        values.extend(result["valuesOT"])
        if best_value is None or result["finalVal"] > best_value:
            best_value = result["finalVal"]
            best_state = result["final"]

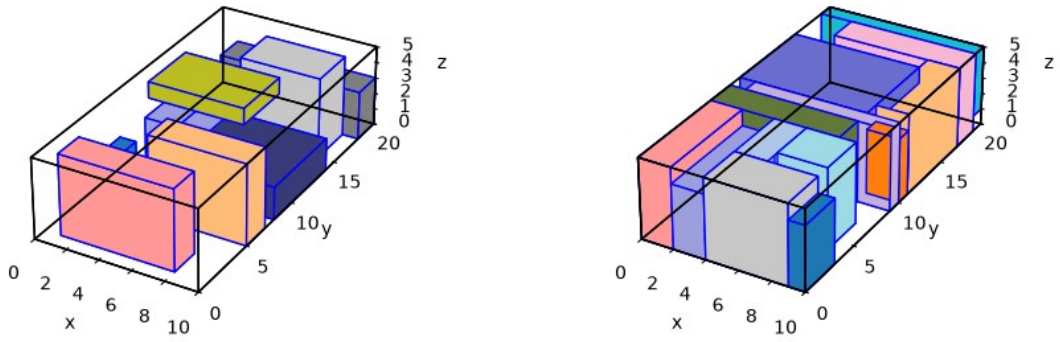
    return {
        "variant": "random_restart",
        "objective": objective,
        "seed": seed,
        "initial": first_initial,
        "final": best_state,
        "finalVal": best_value,
        "realValue": rawValue(best_state, packages),
        "duration": time.perf_counter() - start,
        "valuesOT": values,
        "iterations": sum(iterations_per_restart),
        "max_iter": maxIterPerRestart,
        "max_restart": maxRestart,
        "restarts": restarts,
        "iterations_per_restart": iterations_per_restart,
        "restart_final_values": restart_values
    }
```

Hasil tiga run:

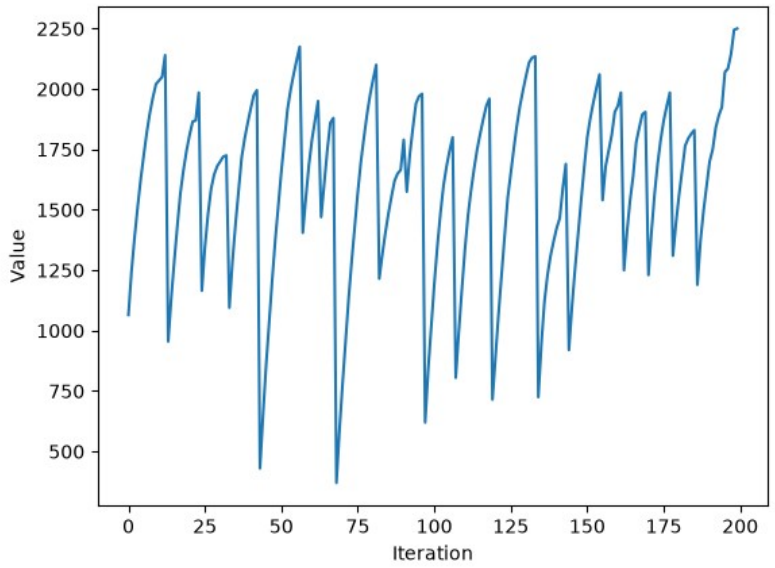
*Tabel 4. Hasil eksperimen random restart hill climbing (maxRestart = 20).*

| Run (seed) | Nilai akhir | Durasi (detik) | Total iterasi | Restart | Iterasi per restart |
|---|---|---|---|---|---|
| 1 | 2150 | 2,169 | 171 | 20 | 4–13 |

| | | | | | |
|---|---|---|---|---|---|
| 2 | 2250 | 2,182 | 180 | 20 | 4–14 |
| 3 | 2250 | 2,035 | 183 | 20 | 2–15 |




*Gambar 7. State awal (kiri) dan state akhir (kanan) random restart run 2 (nilai 2250).*




*Gambar 8. Nilai objective terhadap iterasi lintas seluruh restart (random restart run 2).*

Random restart memberi hasil terbaik sekaligus paling konsisten: 2150, 2250, dan 2250. Nilai akhir per restart pada run 2 berkisar 1690–2250, memperlihatkan variansi kualitas local optimum antar initial state yang besar; pengulangan pencarian dari banyak titik awal efektif menemukan lembah yang lebih tinggi tanpa mengubah mekanisme

langkahnya. Dengan rata-rata hanya sekitar 9 iterasi per restart, total durasinya tetap moderat (≈ 2,1 detik).

### Simulated Annealing

Simulated annealing pada src/simulated_annealing/sa.py memakai representasi state yang hanya memuat paket di dalam truk. Neighbor dibangkitkan acak oleh ranbor dengan tiga jenis langkah (swap, move yang mencakup pemasukan/pengeluaran paket, dan rotate) dan hanya kandidat valid pertama yang dikembalikan. Langkah yang menaikkan nilai selalu diterima; langkah yang menurunkan nilai diterima dengan probabilitas exp(ΔE/T) melalui fungsi acc, sehingga pencarian dapat keluar dari local optimum. Suhu awal T0 = 250 didinginkan secara geometrik dengan faktor 0,9999 hingga Tmin = 10, dengan batas 40.000 iterasi (dalam praktik proses berhenti sekitar 32.000 iterasi saat suhu mencapai Tmin).

*src/simulated_annealing/sa.py — acc, ranbor, Run*

```
def acc(delta, T):
    return math.exp( delta / T)

def ranbor(state, packages, truck, t=50):
    W, L, H = truck["dimensions"]
    aksis = {"x": (1, 2), "y": (0, 2), "z": (0, 1)}
    for _ in range(t):
        move = random.choice(["swap", "move", "rotate"] if state else ["move"])
        new = fotokopi(state)
        if move == "swap":
            a = random.choice(list(state))
            b = random.choice([id for id in packages if id != a])
            if b in state:
                new[a]['position'], new[b]['position'] = state[b]['position'], state[a]
['position']
            else:
                new[b] = state[a]
                del new[a]
        elif move == "move":
            id = random.choice(list(packages))
            if id in state and random.random() < 0.51:
                del new[id]
            else:
                orient = state[id]["orientation"] if id in state else random.sample((0, 1, 2), 3)
                w, l, h = dims(id, orient, packages)
                new.pop(id, None)
                floor = [z for z in possible_height(new, packages) if z + h <= H]
                if not floor or w > W or l > L:
                    continue
                pos = (random.randint(0, W - w), random.randint(0, L - l), random.choice(floor))
                new[id] = {"position": pos, "orientation": orient}
        else:
            id = random.choice(list(state))
            i, j = aksis[random.choice("xyz")]
            orient = list(state[id]["orientation"])
            orient[i], orient[j] = orient[j], orient[i]
            new[id] = {"position": state[id]["position"], "orientation": orient}
        if state_valid(new, packages, truck):
            return new
    return None

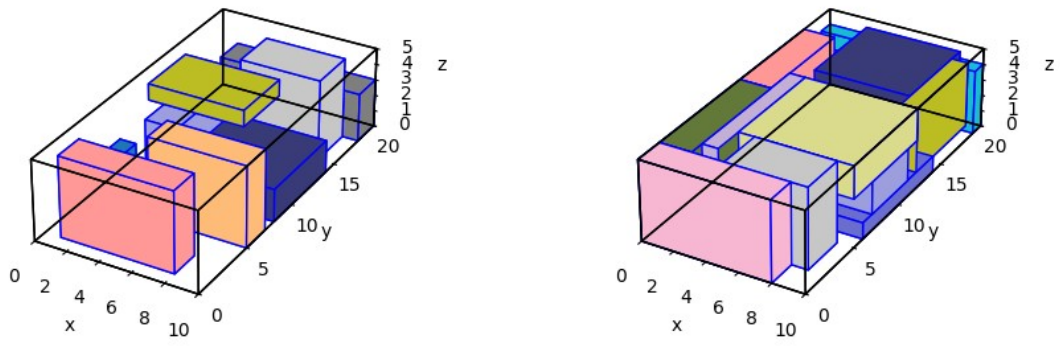
def Run(packages, truck, TStart=250, cooling=0.9999, Tmin=10, t=40000, seed=None):
    random.seed(seed)
    start = time.perf_counter()
    initial = cur = initState(packages, truck)
```

```
    curVal = truckValue(cur, packages)
    T = TStart
    values, probs = [], []
    maxVal, noInc, stuck = curVal, 0,0
    for _ in range(t):
        if T < Tmin:
            break
        new = ranbor(cur, packages, truck)
        prob = None
        if new is not None:
            delta = truckValue(new, packages) - curVal
            prob = acc(delta,T) if delta < 0 else None
            if delta >= 0 or random.random() < prob:
                cur, curVal = new, curVal + delta
        values.append(curVal)
        probs.append(prob)
        T *= cooling
        if curVal> maxVal:
            maxVal, noInc = curVal,0
        else:
            noInc += 1
            if noInc % (t // 50) == 0: stuck +=1
    return {
        "initial": initial, "final": cur, "finalVal": curVal,
        "duration": time.perf_counter() - start,
        "valuesOT": values, "probs": probs, "stuck": stuck
    }
```

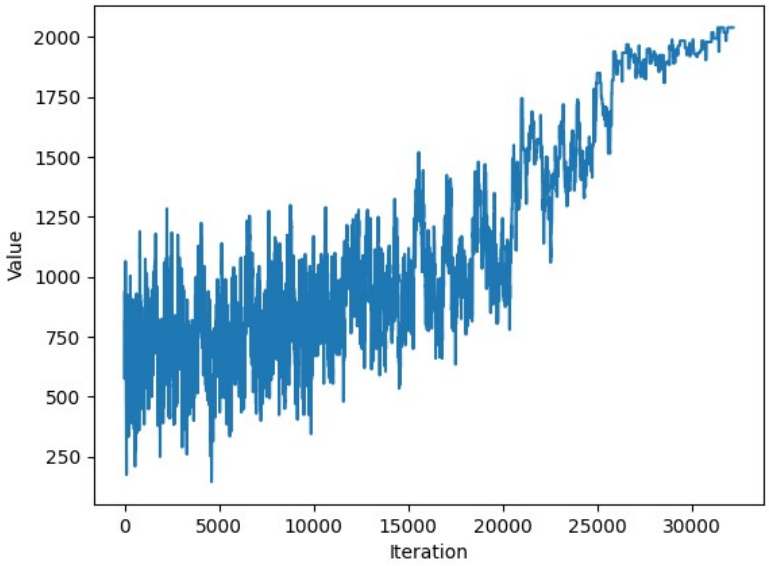
Hasil tiga run beserta frekuensi stuck:

*Tabel 5. Hasil eksperimen simulated annealing (T0 = 250, cooling = 0,9999, Tmin = 10).*

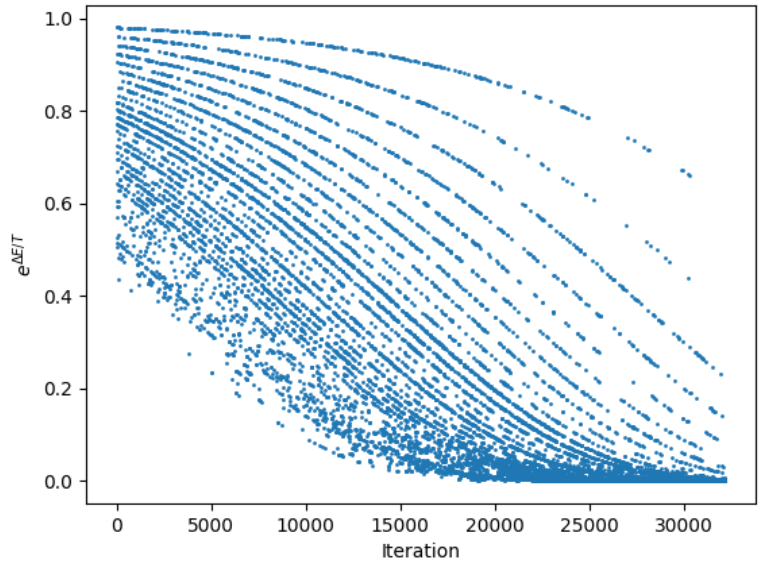
| Run (seed) | Nilai akhir | Durasi (detik) | Frekuensi stuck |
|---|---|---|---|
| 1 | 2030 | 3,302 | 33 |
| 2 | 2040 | 2,866 | 34 |
| 3 | 1940 | 3,071 | 35 |




*Gambar 9. State awal (kiri) dan state akhir (kanan) simulated annealing run 2 (nilai 2040).*




*Gambar 10. Nilai objective terhadap iterasi simulated annealing run 2.*




*Gambar 11. Plot exp(ΔE/T) untuk setiap langkah yang menurunkan nilai (simulated annealing run 2).*

Nilai akhir SA (1940–2040) konsisten dan lebih baik daripada stochastic hill climbing, tetapi masih di bawah random restart. Durasi ≈ 3 detik merupakan konsekuensi dari puluhan ribu iterasi. Plot penerimaan menunjukkan nilai exp(ΔE/T) menyusut seiring suhu menurun: pada awal pencarian langkah penurunan nilai hampir selalu diterima (fase eksplorasi), lalu penerimaan menjadi semakin jarang (fase eksploitasi). Frekuensi stuck dihitung dengan menambah pencacah setiap 800 iterasi tanpa perbaikan nilai terbaik; nilai 33–35 pada ketiga run menunjukkan pencarian memang lama melewati

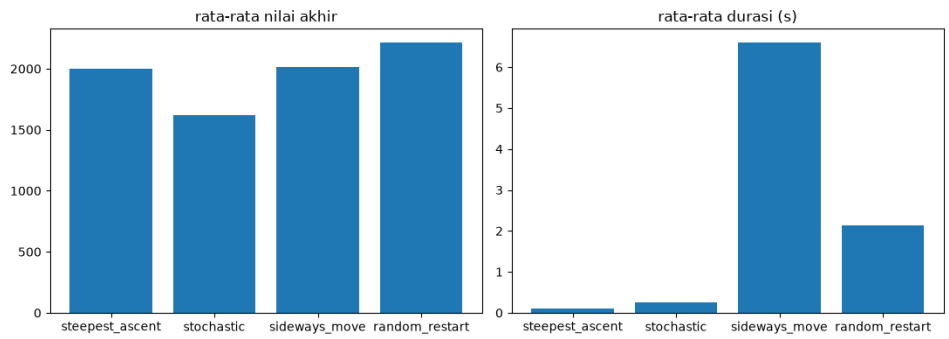
plateau, namun kemampuan menerima langkah sampingan menjaga hasil antar run tetap stabil.

### Analisis Hasil Eksperimen

Ringkasan perbandingan seluruh algoritma berdasarkan tiga run yang dilakukan:

*Tabel 6. Perbandingan hasil eksperimen seluruh algoritma.*

| Algoritma | Nilai terbaik | Rata-rata nilai | Durasi rata-rata (detik) | Rentang nilai |
|---|---|---|---|---|
| Steepest ascent | 2140 | 1996,7 | 0,099 | 1815–2140 |
| Stochastic | 1710 | 1623,3 | 0,256 | 1550–1710 |
| Sideways move | 2140 | 2015,0 | 6,605 | 1815–2140 |
| Random restart | 2250 | 2216,7 | 2,129 | 2150–2250 |
| Simulated annealing | 2040 | 2003,3 | 3,080 | 1940–2040 |




*Gambar 12. Rata-rata nilai akhir dan rata-rata durasi tiap varian Hill Climbing.*

Kedekatan terhadap global optimum. Optimum global persis tidak diketahui, namun batas atas yang valid dapat dihitung dengan mengabaikan geometri penempatan: total value seluruh paket adalah 3420, tetapi total berat 558 melebihi kapasitas 500 dan total volume 1395 melebihi volume peti kemas 1000 sehingga tidak semua paket dapat diangkut. Seleksi subset terbaik dengan kendala berat saja memberi batas 3105, sedangkan dengan kendala berat sekaligus volume memberi batas 2655. Solusi terbaik yang ditemukan (2250) mencapai sekitar 84,7% dari batas 2655; selisihnya berasal dari kendala geometri 3D, aturan support, dan larangan fragile yang tidak diperhitungkan pada batas tersebut.

Perbandingan antar algoritma. Random restart unggul pada nilai akhir dan konsistensi; steepest ascent dan sideways move menempati peringkat berikutnya (2140), simulated

annealing stabil pada 2040, sedangkan stochastic terendah (1710). Pola ini konsisten dengan sifat masing-masing algoritma: steepest ascent terjebak pada local optimum pertama namun sangat cepat; sideways move hanya membantu bila plateau mengarah ke lembah yang lebih baik; random restart mengulang pencarian dari banyak titik awal sehingga peluang menemukan lembah tinggi meningkat; dan simulated annealing menerima langkah memburuk secara probabilistik sehingga tidak terjebak total, meski jadwal pendinginan tetap membatasi eksplorasi.

Durasi. Steepest ascent (≈ 0,1 detik) dan stochastic (≈ 0,26 detik) nyaris instan; random restart ≈ 2,1 detik karena menjalankan 20 pencarian; simulated annealing ≈ 3,1 detik untuk puluhan ribu iterasi; dan sideways move paling lambat (≈ 6,6 detik) karena setiap iterasi mengevaluasi banyak kandidat swap-orientasi. Untuk kebutuhan demo, random restart memberi nilai terbaik dengan durasi yang masih wajar.

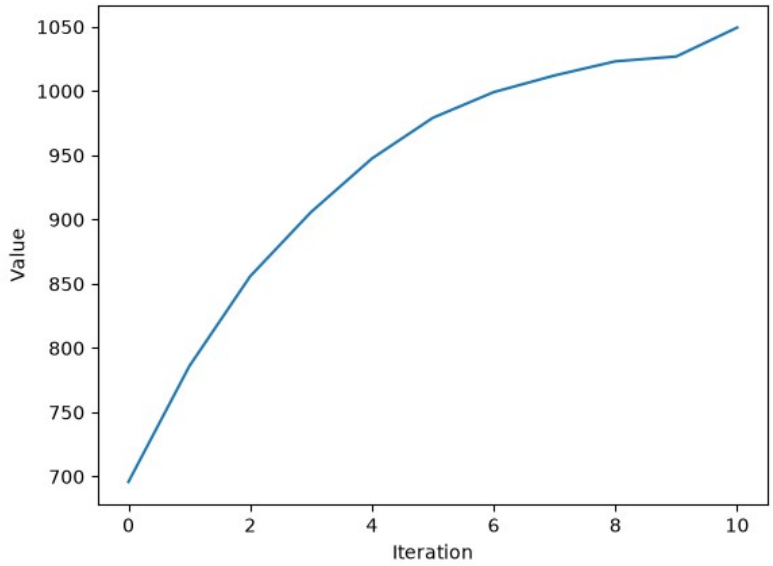
Konsistensi. Random restart dan simulated annealing sangat stabil (rentang 100), steepest ascent dan sideways move memiliki rentang hingga 325 karena kualitas local optimum bergantung pada initial state, sedangkan stochastic berada di antaranya dengan rentang 160.

**Bonus: Objective Function Alternatif (Urgency)**

Selain objective wajib (memaksimalkan value), diimplementasikan objective alternatif urgency dengan skor paket = value ÷ eta: paket yang lebih mendesak (eta kecil) memperoleh skor lebih tinggi. Steepest ascent dijalankan kembali dengan objective ini pada tiga seed yang sama.

*Tabel 7. Perbandingan hasil steepest ascent dengan objective value dan urgency.*

| Run (seed) | Nilai akhir (objective value) | Nilai akhir (objective urgency) | Skor urgency |
|---|---|---|---|
| 1 | 2035 | 1975 | 1319,42 |
| 2 | 2140 | 2010 | 1049,42 |
| 3 | 1815 | 1975 | 1096,92 |




*Gambar 13. Nilai objective terhadap iterasi pada objective urgency (steepest ascent run 2).*

Objective alternatif mengubah pilihan swap yang diambil. Run 3 justru memperoleh nilai riil lebih tinggi (1815 → 1975) karena prioritas paket mendesak mengarah ke konfigurasi yang lebih padat, sedangkan run 1 dan 2 menurun sedikit sebagai konsekuensi trade-off antara value dan ETA. Yang terpenting, objective function dapat ditukar tanpa mengubah algoritma, constraint, maupun representasi state — menunjukkan fleksibilitas rancangan program. Sebagai catatan bonus lainnya, keempat varian Hill Climbing telah diimplementasikan seluruhnya.

# III. Kesimpulan dan Saran

Implementasi local search untuk permasalahan pengisian peti kemas truk tiga dimensi telah selesai dan seluruh state yang dihasilkan pada eksperimen memenuhi constraint (batas peti kemas, tabrakan antar paket, kapasitas berat, support, dan aturan fragile). Program Python mencakup empat varian Hill Climbing, Simulated Annealing, visualisasi state tiga dimensi, serta plot nilai objective dan probabilitas penerimaan simulated annealing.

Random restart hill climbing mencapai hasil terbaik (2250) dan paling konsisten (2150 –2250), diikuti steepest ascent dan sideways move (maksimum 2140), simulated annealing (2040), dan stochastic hill climbing (1710). Terhadap batas atas longgar 2655 (seleksi subset dengan kendala berat dan volume, tetap mengabaikan geometri penempatan), solusi terbaik yang ditemukan setara dengan sekitar 84,7%. Dari sisi durasi, steepest ascent dan stochastic nyaris instan, random restart dan simulated

annealing berada di kisaran 2–3 detik, sedangkan sideways move paling lambat karena biaya evaluasi kandidatnya besar.

- Saran pengembangan: memperbesar anggaran pencarian (maxSideways, maxRestart, jumlah percobaan stochastic) untuk mengamati titik jenuh peningkatan nilai.
- Saran efisiensi: mempercepat sideways move dengan evaluasi inkremental delta skor, karena bottleneck utamanya adalah pemindaian kandidat pada setiap iterasi.
- Saran parameter: menala parameter simulated annealing (T0, faktor pendinginan, Tmin) dan mengukur kualitas solusi terhadap waktu pencarian untuk menemukan jadwal yang lebih baik.
- Saran pengujian: menambah variasi ukuran instans (jumlah paket dan dimensi truk) untuk menguji skalabilitas serta mengembangkan dukungan multi-truk sebagai lanjutan bonus.

# IV. Pembagian Tugas

Pembagian tugas antar anggota kelompok adalah sebagai berikut:

*Tabel 8. Pembagian tugas anggota kelompok.*

| **Anggota** | **NIM** | **Kontribusi** |
|---|---|---|
| Mohammad Najmutstsaqib | 13223040 | Inisialisasi pemodelan 3D packing dan generator initial state, validasi constraint dasar, serta pengembangan Genetic Algorithm beserta eksperimennya. |
| Ega Luthfi Rais | 13524115 | Implementasi empat varian Hill Climbing dan heuristik penempatan paket, eksekusi dan visualisasi eksperimen Hill Climbing (tiga run per varian), bonus objective alternatif (urgency), penyusunan bab Pembahasan dan analisis laporan. |
| Ahmad Fauzan Putra | 13524141 | Implementasi Simulated Annealing, fungsi visualisasi umum (state 3D dan kurva nilai), README dan struktur awal repositori, dukungan eksperimen Simulated Annealing. |

# V. Referensi

[1] M. L. Khodra, “Beyond Classical Search: Classical vs Local Search,” YouTube, 2021. [Video]. Diakses: 16 September 2026.

[2] M. L. Khodra, “Beyond Classical Search: State (Value, Neighbor),” YouTube, 2021. [Video]. https://www.youtube.com/watch?v=_6rla37QorM. Diakses: 16 September 2026.

[3] M. L. Khodra, “Beyond Classical Search: Hill-climbing Search,” YouTube, 2021. [Video]. https://www.youtube.com/watch?v=L0b0CN3Fds8. Diakses: 16 September 2026.

[4] M. L. Khodra, “Beyond Classical Search: Simulated Annealing,” YouTube, 2021. [Video]. Diakses: 16 September 2026.

[5] M. L. Khodra, “Beyond Classical Search: Genetic Algorithm,” YouTube, 2021. [Video]. Diakses: 16 September 2026.

[6] Python Software Foundation, “The Python Tutorial,” https://docs.python.org/3/tutorial/. Diakses: 19 Oktober 2026.

[7] Python Software Foundation, “pathlib — Object-oriented filesystem paths,” https://docs.python.org/3/library/pathlib.html. Diakses: 19 Oktober 2026.

[8] Matplotlib Development Team, “mplot3d toolkit,” https://matplotlib.org/stable/api/toolkits/mplot3d.html. Diakses: 19 Oktober 2026.

[9] GeeksforGeeks (referensi fungsi list, sort, random, raise, dan infinity yang tercantum pada komentar kode), https://www.geeksforgeeks.org/. Diakses: 19 Oktober 2026.

[10] Stack Overflow, “How to raise a ValueError in Python,” https://stackoverflow.com/questions/4393268. Diakses: 19 Oktober 2026.

# VI. Lampiran Form Penggunaan Generative AI

**FORMAT DEKLARASI PENGGUNAAN AI**
**SURAT PERNYATAAN PENGGUNAAN AI**

Dengan ini, saya/kami*:
Nama/NIM : Mohammad Najmutstsaqib 13223040; Ega Luthfi Rais 13524115; Ahmad Fauzan Putra 13524141 Fakultas/Sekolah : Sekolah Teknik Elektro dan Informatika (STEI)
Kode Mata Kuliah : IF3170
Nama Mata Kuliah : Inteligensi Artifisial
Judul Tugas/Proposal : Tugas Besar 1 — Searching: Optimasi Pengisian Peti Kemas Truk dengan Algoritma Local Search

Menyatakan bahwa kami tidak menggunakan AI dalam pengerjaan maupun penyusunan luaran

tugas pada mata kuliah yang tertulis di atas.

Jika Ya, maka alat AI/kecerdasan buatan yang digunakan adalah:

| Lingkup Pekerjaan | Digunakan? | Tingkat Penggunaan |
|---|---|---|
| **Pemeriksaan Ejaan:** menggunakan tools seperti Grammarly, Paperpal, Deepl, Quillbot, Grammarbot, LanguageTool, ProWritingAid, ChatGPT, Google Gemini, atau sejenisnya. | Tidak | NO AI |
| **Pembuatan Teks:** menggunakan tools seperti ChatGPT, Gemini, Paperpal, Copilot, GrammarlyGO, WordAI, WriteSonic, Jasper, Jenni AI, atau sejenisnya. | Tidak | NO AI |
| **Bantuan Diskusi dalam Penyusunan Konten:** menggunakan tools seperti ChatGPT, Gemini, Copilot, Perplexity, Jenni AI, Zoom-Companion, atau sejenisnya. | Tidak | NO AI |
| **Pembuatan Gambar, Video, dan/atau Grafik:** menggunakan tools seperti Craiyon, DALL-E, Midjourney, Stable Diffusion, Microsoft Designer, Gemini, Canva AI, atau sejenisnya. | Tidak | NO AI |
| **Lainnya, Jelaskan:** | | |

Kami juga menyatakan bahwa setiap penggunaan AI/kecerdasan buatan yang dilakukan pada mata kuliah tersebut telah dinyatakan di dalam surat ini.

(tempat), (tanggal)

(tanda tangan)

(Nama Lengkap)

NIM